Peak ETL

PostGIS + Martin: Building Open-source Mapping Applications

Jason Holden

CEO, PRESENTOR

JASON@PEAKETL.COM | (918) 409-CODE (2633) | (719) 310-5240

About
Me

The Story (so far):

I started working with computers at the age of 13. Growing up in a small town in eastern Oklahoma, I acquired a reputation as the "computer kid." Shortly after graduating high school, I got my first software development job working with Visual FoxPro. Since then, I've used numerous other languages and platforms to develop countless database applications and services. My newest endeavor is Peak ETL. At Peak ETL, we're creating a subscription-based platform to aggregate and analyze public oil and gas data and deliver it to industry professionals for use in their operations.

Jason Holden
CEO & Founder

What We're Building

Overview

JSON
crimes.json

PYTHON
load_data.py

POSTGIS
Spatial Database

Martin
Tile Server

MapLibre GL JS
Browser Map

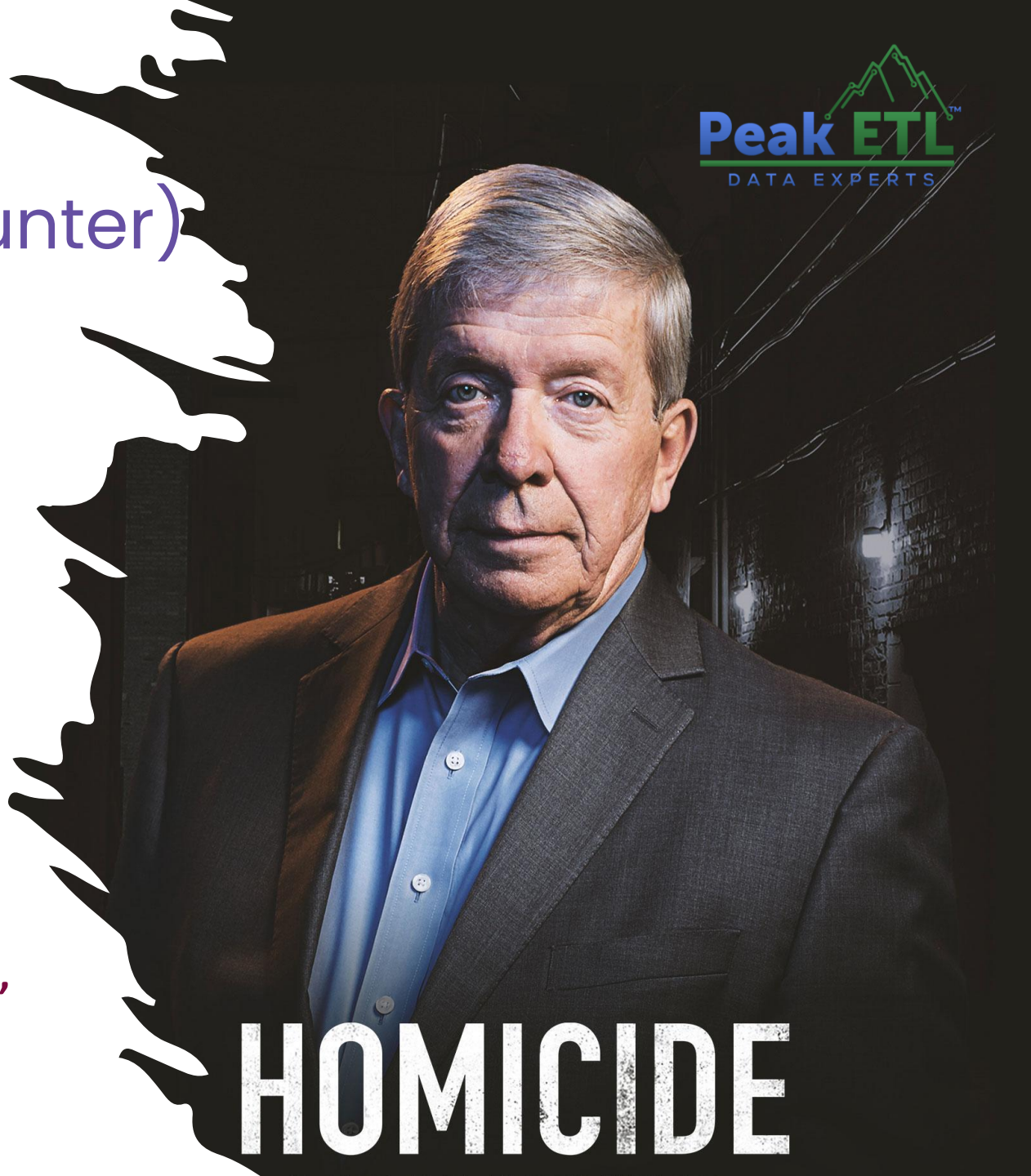
Podman
CLI starts entire stack

Vector Tiles
MVT – fast at every zoom level

Free / OSS
0$ / No API Keys

About this dataset
Lt. Joe Kenda (Homicide Hunter)

1. **1973** – Joins Colorado Springs PD
2. **1977** – Promoted to Homicide Det. after solving an "unsolvable" double shooting
3. **1977 - 1996** – Works 387 homicide cases, closes **356 (92%)**
4. **1996** – Retires as Commander, Major Crimes Unit
5. **2011 - 2020** – *Homicide Hunter* on Investigation Discovery **144 episodes, 9 seasons**



PostGIS
Concepts

PostGIS turns PostgreSQL into a spatial database for maps and GIS.

Geometry Types

- POINT (lon lat) – A Single Location
- LINESTRING – roads, paths
- POLYGON – neighborhoods, parcels
- GEOMETRY – Any of the above

SRID / Coordinate Systems

- 4326 = WGS 84 (GPS lat/lon) – Use this!
- 3857 = Web Mercator (pixel coordinates)
- Martin/MapLibre handle reprojection

Spatial Index

- GiST index on the geom column
- Required for tile serving performance
- USING GIST (geom)

Key Functions:

```sql
-- Create a point from lon, lat
SELECT ST_MakePoint(-104.82, 38.83);

-- Create point + set SRID at once
SELECT ST_SetSRID(
  ST_MakePoint(-104.82, 38.83),
  4326
);

-- EWKT shorthand (what we use)
SELECT ST_GeomFromEWKT(
  'SRID=4326;POINT(-104.82 38.83)'
);

-- Distance in meters (reproject to 3857)
SELECT ST_Distance(
  ST_Transform(a.geom, 3857),
  ST_Transform(b.geom, 3857)
) AS dist_meters;
```

Martin
Concepts

Open-source PostGIS vector tile server

Vector Tiles (MVT)

- Mapbox Vector Tile format (.mvt)
- Binary, compressed – very fast to transfer
- Tiles are /{z}/{x}/{y} – zoom / column / row
- Browser renders them with WebGL (MapLibre)
- Styling happens in the browser – no baked into the tile
- Cacheable (CDN)

Martin Endpoints

- /crimes ⇨ TileJSON metadata
- /crimex/{z}/{x}/{y} ⇨ actual tile
- /catalog ⇨ lists all sources
- /health ⇨ health check

./containers/martin/martin.yaml

```yaml
# PostgreSQL connection
postgres:
  connection_string: ${DATABASE_URL}

  auto_publish:
    tables: true
    from_schemas: public

# Martin web server settings
listen_addresses: "0.0.0.0:3000"
```

MapLibre GL JS
Concepts

Open-source JS library for WebGL-based vector map rendering

History

- Mapbox GL JS fork
- Maintained by the MapLibre Organization
- Backed by AWS, Microsoft, and Meta

Client-side Browser

- WebGL rendering (GPU acceleration)
- Consumes vector tiles directly
- Styling and theming defined in JSON style specification
- Dynamic styling
- Vanilla JS
- Fully open-source

./app.js

```
const MARTIN_URL  = "http://localhost:3000";          // Martin tile server
const TABLE       = "crimes";                         // PostGIS table name
const MAP_CENTER  = [-104.8214, 38.8339];             // Colorado Springs
const MAP_ZOOM    = 11;

// Base map
const BASEMAP_STYLE = "https://tiles.openfreemap.org/styles/dark";

// State
let map;
let allCases  = [];     // master list from Martin TileJSON
let popup     = null;
let activeId  = null;
let filters   = { season: "all", solved: "all", yearMax: 1996 };

// Fetch case list from Martin, then init the map
async function bootstrap() {
    // Martin exposes a TileJSON endpoint for each table.
    // We also fetch a GeoJSON fallback to populate the sidebar.
    // (In production you'd use your own API; here we embed crimes.json)
    const resp = await fetch("crimes.json");
    const data = await resp.json();
    allCases   = data.cases;

    renderSidebar(allCases);
    updateStats(allCases);
    initMap();
}
```

./index.html

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Homicide Hunter — Colorado Springs Crime Map</title>

        <!-- MapLibre GL JS (open-source, no API key required) -->
        <link rel="stylesheet" href="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.css">
        <script src="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.js"></script>

        <link rel="stylesheet" href="style.css">
    </head>
    <body>
        <header>
            <span class="badge">🔍</span>
            <div>
                <h1>Homicide Hunter — <span>Colorado Springs</span></h1>
                <div class="subtitle">Lt. Joe Kenda &middot; 1977&ndash;1996 &middot; PostGIS + Martin + MapLibre GL</div>
```

Stack Configuration

Stack is started from the CLI

Startup Shell Script

- `start.sh`
- Stop running containers
- Build fresh containers
- Start PostGIS
- Import data (`load_data.py`)
- Start Martin
- Start Nginx

./containers/postgis/Containerfile

```
FROM docker.io/postgis/postgis:18-3.6

COPY schema.sql /docker-entrypoint-initdb.d/kenda_cases_schema.sql

EXPOSE 5432
```

./load_data.py

```
import json
import os
import sys
import psycopg2
from psycopg2.extras import execute_values

# ── Connection ──────────────────────────────────────────────────
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://postgres:postgres@localhost:5432/postgres"
)

# ── Load the JSON dataset ───────────────────────────────────────
def load_cases(json_path: str = "crimes.json") -> list[dict]:
    with open(json_path) as f:
        data = json.load(f)
    return data["cases"]
```

./containers/martin/Containerfile

```
FROM ghcr.io/maplibre/martin:latest

COPY martin.yaml /config.yaml

EXPOSE 3000

CMD ["--config", "/config.yaml"]
```

./containers/nginx/Containerfile

```
FROM docker.io/nginx:stable-alpine3.23

COPY nginx.conf /etc/nginx/nginx.conf
```

Dataset Configuration

JSON File

- Easy to share
- Versioning
- Easy to load

./crimes.json

```
{
  "cases": [
    {
      "id": 2,
      "episode":   "S01E02",
      "title":     "Evil Eyes",
      "victim":    "Richard Gray",
      "year":      1986,
      "crime_type":"Homicide",
      "weapon":    "Blunt force / eye trauma",
      "location_name": "East Side Office Park",
      "neighborhood":  "East Colorado Springs",

      "lat":  38.8401,   // ← latitude  (y)
      "lon": -104.7412,  // ← longitude (x) — note negative (west)

      "solved":  true,
      "season":  1,
      "description": "July 3, 1986: A security guard finds a woman
        covered in blood next to a dead man at a vacant office park..."
    },
    // ... 26 more cases
  ]
}
```

Database Schema

PostGIS Schema

- `geom` (The "magic" column – PostGIS Point in WGS 84)
- Index for tile lookup efficiency
- Index on Season/Year for filtering efficiency

./containers/postgis/schema.sql

```sql
-- Enable the PostGIS extension (adds geometry types + functions)
CREATE EXTENSION IF NOT EXISTS postgis;

-- Our crimes table
CREATE TABLE IF NOT EXISTS crimes (
    id            SERIAL PRIMARY KEY,
    episode       TEXT,
    title         TEXT NOT NULL,
    season        INTEGER,
    victim        TEXT,
    year          INTEGER,
    crime_type    TEXT,
    weapon        TEXT,
    solved        BOOLEAN DEFAULT true,
    description   TEXT,
    location_name TEXT,
    neighborhood  TEXT,
    lat           DOUBLE PRECISION,
    lon           DOUBLE PRECISION,

    -- ↓ The magic column – PostGIS Point in WGS 84
    geom  GEOMETRY(Point, 4326)
);

-- Spatial index – Martin needs this to tile efficiently
CREATE INDEX IF NOT EXISTS crimes_geom_idx
    ON crimes USING GIST (geom);

-- Regular indexes for our filter queries
CREATE INDEX IF NOT EXISTS crimes_year_idx   ON crimes (year);
CREATE INDEX IF NOT EXISTS crimes_season_idx ON crimes (season);
```

Data
Loader

Python Data Loader

- Executed on startup
- Loads `crimes.json`
- `INSERT` each case JSON `dict` into `crimes` table

./load_data.py

```python
#!/usr/bin/env python3

import json, os, sys, psycopg2
from psycopg2.extras import execute_values

# Connection string
DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/postgres")

# JSON file
JSON_PATH = "crimes.json"

# Load the JSON dataset
def load_cases(json_path:str = JSON_PATH) -> list[dict]:
    with open(json_path) as f:
        data = json.load(f)
    return data["cases"]

# Insert rows into PostGIS
INSERT_SQL = """
    INSERT INTO crimes (
        episode
        ,title
        ,season
        ,victim
        ,year
        ,crime_type
        ,weapon
        ,solved
```

Key
Take Aways

Going Further

- **PostGIS**
  - ST_Buffer – radius around a point
  - ST_ClusterKMeans – cluster crimes
  - ST_Hexagon – hexbin aggregation
  - PostGIS raster support (elevation)
  - Spacial queries – PostGIS In Action (Manning Press)
  - pg_bouncer for connection pooling
- **Martin**
  - Server SQL functions as tile sources
  - MbTiles file support (offline)
  - Sprint + glyph config for symbols
  - CDN tile caching
  - Qgis – Open-source Desktop special visualization (qgis.org)
- **MapLibre**
  - GeoJSON sources for live data
  - Heatmap layers
  - 3D terrain / extrusions
  - Deck.gl integration

Contact Us

GitHub

https://github.com/peaketl/open-source-mapping-presentation

CONTACT

(719) 310-5240
(918) 409-CODE (2633)

www.peaketl.com
jason@peaketl.com